

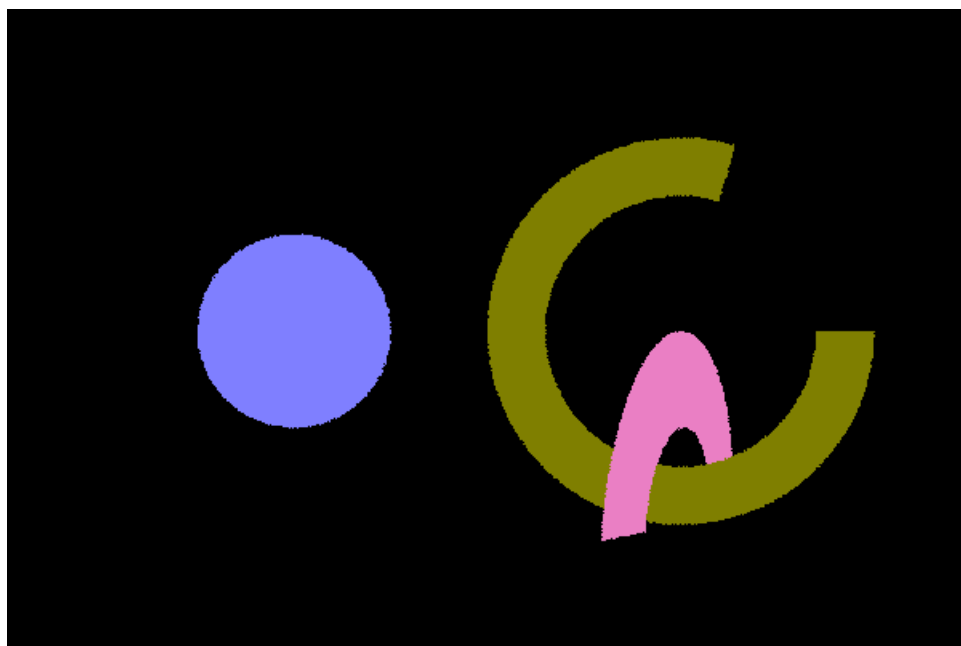
图形绘制技术 2026 Lab1

闵文楷 241870230

一.物体求交

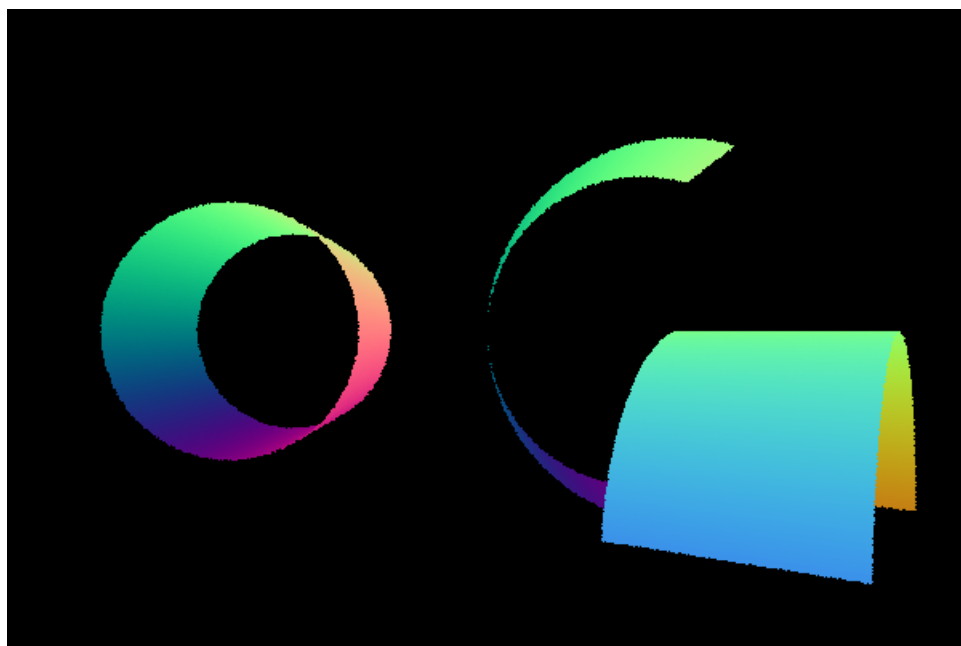
1.圆环:

令光线参数方程 z 坐标为 0 即可求出时间 t , 进而求出交点; 法线只需取 $+z$ 方向单位向量即可



2.圆柱:

令光线方程 x, y 坐标满足圆柱半径, 如果方程有根则依次判断时间 t , 交点 z 坐标以及角度是否符合要求, 保留符合要求的较小的 t ; 法线取所在圆面半径向外方向.



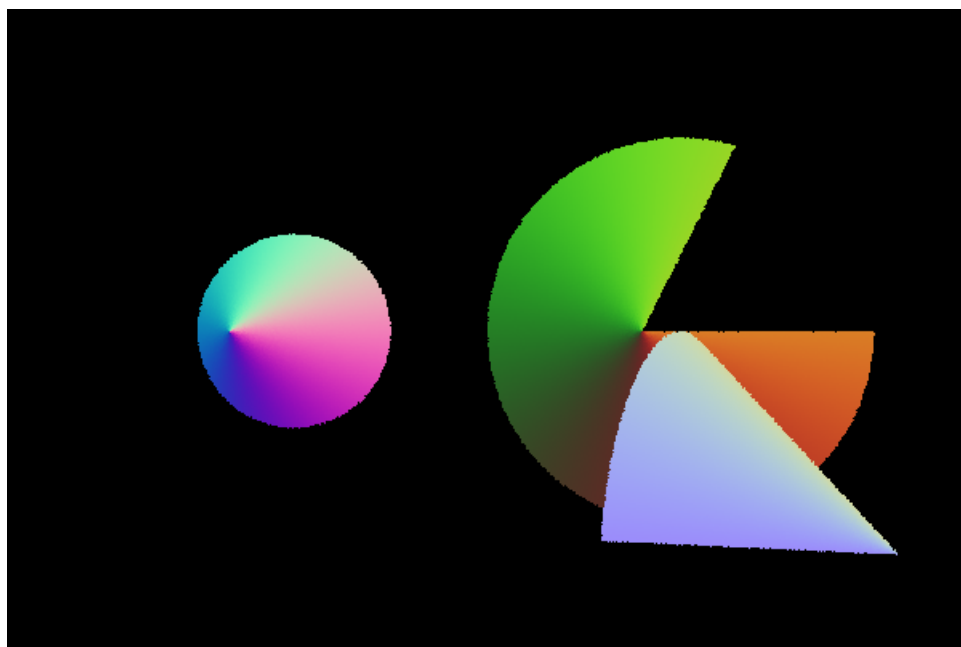
3.圆锥:

联立光线方程和圆锥表面方程, 如果有根, 则依次判断时间 t , 交点 z 坐标以及角度是否符合要求, 保留符合要求的较小的 t ; 法线方向利用几何关系计算.

```

\\ 交点和法线计算逻辑
float phi = u * phiMax;
float theta = v * height;
Point3f position = Point3f(0, 0, theta) + (radius -
(theta/height)*radius) * Vector3f(cos(phi), sin(phi), 0);
intersection->position = transform.toWorld(position);
Point3f K = Point3f(0, 0, height - (height - theta)/(cosTheta *
cosTheta));
Vector3f n = normalize(position - K);
intersection->normal = transform.toWorld(n);

```



二.加速结构

在实现加速结构之前需要先完成 Triangle 求交

1.BVH:

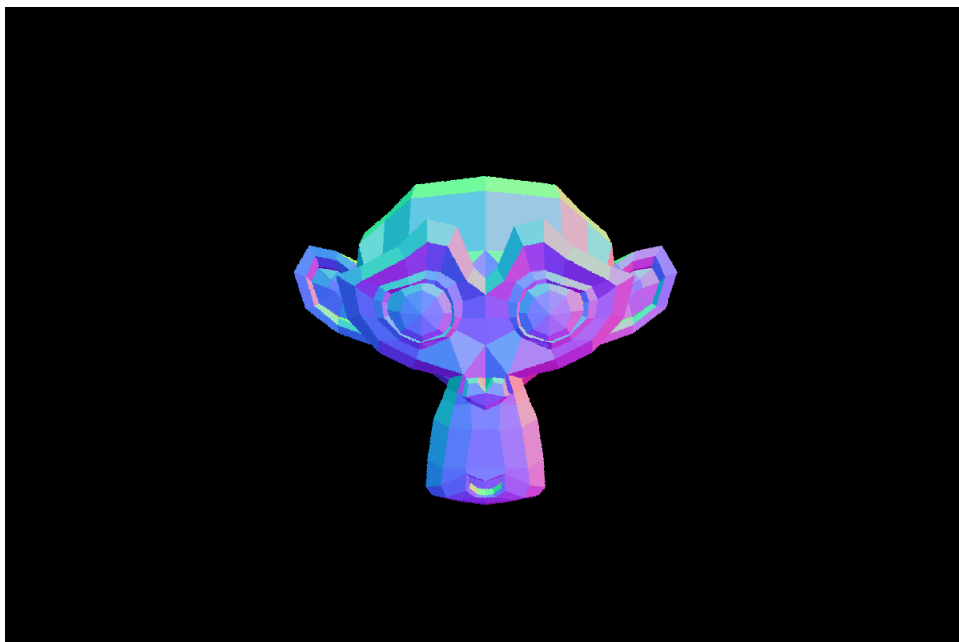
节点结构:

在 **BVHNode** 结构体下定义节点结构, 包括左右子树, 包围盒, 叶子区间, 分割轴

BVH 递归构建:

先初始化每个 shape 的内部加速结构, 以 shape AABB 中心在最长轴上排序并二分; 当节点包含的叶子数低于阈值时停止分割, 构建叶子

BVH 求交: 从根节点开始遍历, 若光线与当前节点 AABB 盒相交, 递归遍历子节点, 到叶子时依次对节点内每个物体求交; 若光线与当前节点 AABB 盒不相交, 直接返回



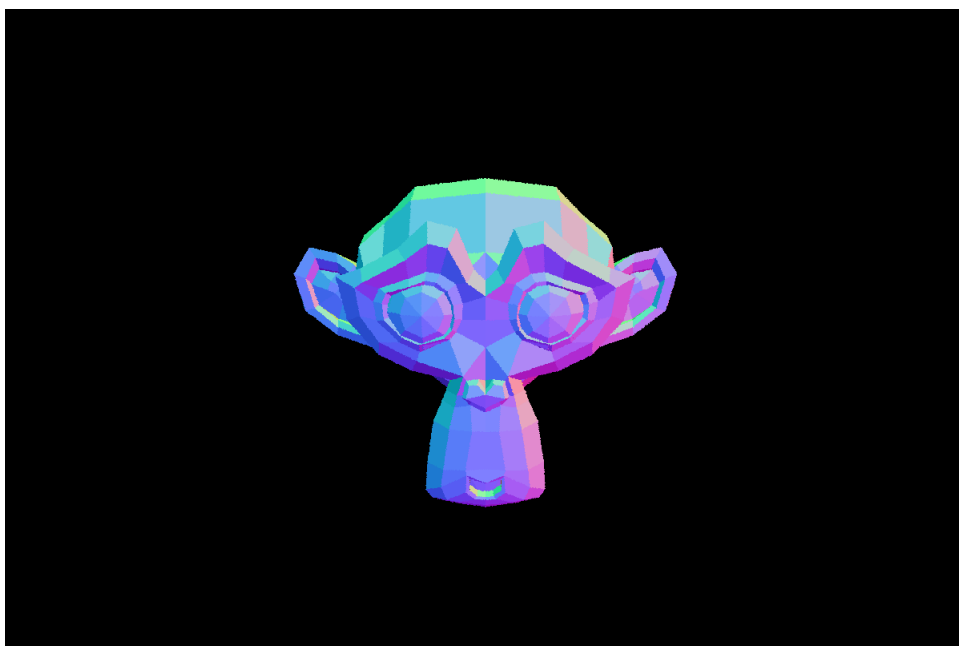
2.Octree:

建树:

在 `recursiveBuild` 方法下完成八叉树的递归构建, 按中心将当前节点分为 8 个子 AABB, 按 `Overlap` 分发 primitive; 若某个子盒与当前节点所有物体都相交, 则停止细分, 当前节点作为叶子.

求交:

从根节点开始遍历, 若光线与当前节点 AABB 盒相交, 递归遍历子节点, 到叶子时依次对节点内每个物体求交; 若光线与当前节点 AABB 盒不相交, 直接返回



对比: Octree 3.24s; BVH 4.01s; linear 过慢, 未跑完, 估计要 300s 左右.

三.景深

定义 `cameraSpaceFocusPoint` 函数, 计算 pFocus

```
static Point3f cameraSpaceFocusPoint(float filmX, float filmY, float
verticalFov,
                                   int filmHeight, float focalDistance)
{
    float tanHalfFov = fm::tan(verticalFov * 0.5f);
    float z = -filmHeight * 0.5f / tanHalfFov;
    Vector3f w = normalize(Vector3f{filmX, filmY, z});
    float t = -focalDistance / w[2];
    return Point3f{w[0] * t, w[1] * t, w[2] * t};
}
```

在`sampleRay`中实现圆盘采样, 并计算采样点和 `pFocus` 确定的光线方向

```
Ray ThinLensCamera::sampleRay(const CameraSample& sample, Vector2f NDC)
const {

    float x = (NDC[0] - 0.5f) * film->size[0] + sample.xy[0],
           y = (0.5f - NDC[1]) * film->size[1] + sample.xy[1];

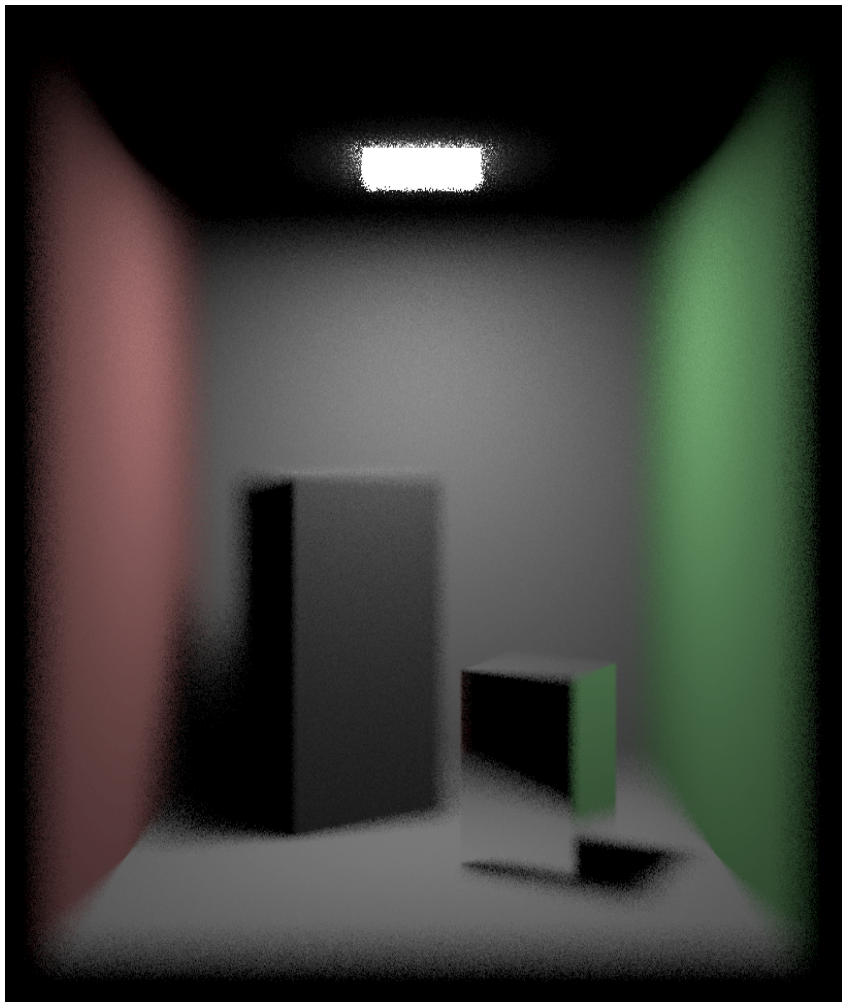
    Point3f pFocus =
        cameraSpaceFocusPoint(x, y, verticalFov, film->size[1],
focalDistance);

    float u1 = sample.lens[0], u2 = sample.lens[1];
    float r = lensRadius * std::sqrt(u1);
    float theta = 2.f * PI * u2;
    float lx = r * fm::cos(theta);
    float ly = r * fm::sin(theta);
    Point3f lens0{lx, ly, 0.f};

    Vector3f dirCam = normalize(pFocus - lens0);
    Point3f originW = transform.toWorld(lens0);
    Vector3f dirW = normalize(transform.toWorld(dirCam));

    return Ray(originW, dirW, tNear, tFar, timeStart);
}
```

在`sampleRayDifferentials`中实现射线微分, 可用于纹理过滤和 Mipmap 选择



focalDistance = 5.7

四.

图15注释,不太理解为什么**焦平面**和**透镜**之间的距离是**焦距**,可能实际指的是**对焦距离**(?)